UC11 · DESENVOLVER ALGORITMOS E PROGRAMAÇÃO ESTRUTURADA

Python para Cibersegurança

Aula final — colocando tudo em prática: Docker, brute force e Wireshark

Técnico em Segurança Cibernética

Hoje é 100% prática

O roteiro de hoje

01 Subir o **banco_web** em Docker — o alvo de hoje

02 Os 3 guias de **brute force** — do simples ao rockyou.txt

03 Ver o ataque com os próprios olhos no **Wireshark**

04 Panorama: até onde o Python vai em cyber

Um combinado antes de começar

O que hoje NÃO é

Uma aula de sintaxe. Ninguém aqui vai sair escrevendo scripts do zero, e tudo bem.

O que hoje É

Rodar ferramentas reais, entender o que elas fazem e enxergar como rede + script + ferramenta se conectam.

Segurança cibernética é uma área enorme. Programar é **uma** ferramenta a mais na caixa — não é o centro dela. O que importa é entender o que as ferramentas fazem, ler o resultado e saber onde procurar quando precisar.

Onde o Python entra na cibersegurança

Automatizar ataques

Brute force, varredura de portas, testes de login em massa — tudo que seria manual e lento vira um loop.

Automatizar defesa

Ler logs, detectar padrões de ataque, disparar alertas — o mesmo tipo de script, do outro lado do muro.

Cola entre ferramentas

Python conversa com APIs, bancos, arquivos e a rede — é o que junta as peças que Nmap, Hydra, Burp e Wireshark deixam soltas.

Repare: em nenhum desses três pontos a pergunta é "o aluno sabe decorar sintaxe Python?" — é "o aluno entende o que o script está tentando fazer contra a rede?"

Os três materiais da trilha

`material_bruteforce`	3 guias práticos: do bruteforce mais simples até o uso do rockyou.txt de verdade. **É o nosso foco de hoje.**
`material_kali`	Ferramentas do Kali Linux: Wireshark, Burp Suite e Hydra vs. Python. **Hoje usamos a parte de Wireshark.**
`material_blackhat`	Baseado no livro Black Hat Python: C2, trojans, keylogger. **Não vamos rodar hoje — é para quem quiser ir além.**

Os três estão no GitHub de vocês, prontos para consulta depois da aula.

BLOCO 01

Preparando o campo de testes

Subindo o banco_web em Docker

Subindo o alvo: docker compose up

```
# dentro da pasta do projeto banco_web
git clone https://github.com/fernandoaaeng/SENAC.git
cd SENAC/banco_web_aula_21_09
docker compose up -d
```

O que sobe

API (FastAPI) na porta **8002**, frontend na porta **8080** e o banco de dados

Como confirmar

docker ps mostra os containers rodando

CHECKPOINT 5 minutos

Antes de seguir, confirme os três itens

- ✓ docker ps mostra os containers do banco_web como "Up"
- ✓ O navegador abre o frontend em http://localhost:8080
- ✓ Anotado o IP/porta da API (8002) que os scripts vão usar

Só avance quando todo mundo estiver com os três itens marcados.

BLOCO 02

Os três guias de brute force

material_bruteforce — do simples ao rockyou.txt

Guia 01

Iniciante — um usuário, uma lista de senhas

Revisa cliente-servidor, HTTP e status codes: **200** deu certo, **401** senha errada.

O script bruteforce_simples.py tem só 18 linhas: abre senhas.txt com open(), testa uma por uma e para com break quando acha.

```
# a ideia central
for senha in lista_de_senhas:
    resposta = requests.post(url, json=
{"senha": senha})
    if resposta.status_code == 200:
        print("Senha encontrada:", senha)
        break
```

EXERCÍCIO 1 · GUIA 01 15 minutos

Rode o primeiro ataque de verdade

1. Abram o material_bruteforce e leiam o 01_GUIA_PRATICO_INICIANTE.md e os exercícios do arquivo 01_EXERCICIOS_INICIANTE.md.

2. Ajustem a URL do script para apontar para o banco_web que subiu no Docker.

3. Rodem python bruteforce_simples.py e observem a senha sendo encontrada.

Terminal

```
$ pip install requests
$ python bruteforce_simples.py
Senha encontrada: 123456
```

Guia 02

Vários usuários, várias senhas

Agora são **dois arquivos**: usuarios.txt e senhas.txt.

O script bruteforce_multi_usuarios.py usa um **loop dentro de outro loop**: para cada usuário, testa todas as senhas.

Aparece também o **403** — usuário existe, mas sem permissão para aquela ação.

```
# a ideia central
for usuario in lista_de_usuarios:
    for senha in lista_de_senhas:
        dados = {"usuario": usuario, "senha": senha}
        resposta = requests.post(url, json=dados)
```

EXERCÍCIO 2 · GUIA 02 15 minutos

Ataque a múltiplos usuários

1. Leiam 02_GUIA_PRATICO_USUARIOS_E_SENHAS.md e um exercício de 02_EXERCICIOS_USUARIOS_E_SENHAS.md.

2. Adicionem os usuários reais do banco_web (ana, bruno, carlos) no usuarios.txt.

3. Rodem bruteforce_multi_usuarios.py e anotem quais combinações deram 200.

O que observar

Quanto mais usuários e senhas, mais tempo o script leva — é **N × M** tentativas.

Guia 03

rockyou.txt — o mundo real

O rockyou.txt tem **14 milhões** de senhas reais vazadas — grande demais para carregar tudo na memória de uma vez.

bruteforce_rockyou.py lê o arquivo linha por linha (streaming), usa encoding="latin-1" e protege cada tentativa com try/except.

```
# protegendo contra falhas de rede
try:
    resposta = requests.post(url, json=dados, timeout=5)
except requests.exceptions.RequestException:
    continue
```

O guia também mostra como calcular a **velocidade** do ataque (tentativas por segundo) e montar um relatório de auditoria no final.

EXERCÍCIO 3 · GUIA 03 15 minutos

Ataque em escala, com relatório

1. Leiam 03_GUIA_PRATICO_ROCKYOU.md — não precisa rodar a wordlist inteira, um recorte de algumas milhares de linhas já serve.

2. Rodem bruteforce_rockyou.py e observem o relatório final: quantas tentativas, quanto tempo, senha encontrada.

3. **Desafio:** derrubem o banco_web no meio da execução e vejam o script sobreviver ao erro de rede.

Resultado esperado

```
Tentativas: 4.812
Tempo: 38.2s (~126/s)
Senha encontrada: senha123
```

BLOCO 03

Vendo o ataque pela rede

material_kali — Wireshark capturando o brute force

O que é o Wireshark

Até agora, o Python **falava** com a rede. O Wireshark deixa a gente **ver** essa conversa acontecendo, pacote por pacote.

É a mesma ferramenta que um analista de blue team usa para descobrir que alguém está tentando invadir um sistema.

O script Python envia

```
requests.post(url, json=dados)
```

↓

O Wireshark mostra

```
POST /login HTTP/1.1 ...
{"usuario":"ana","senha":"123456"}
```

Por que isso importa: HTTP vs. HTTPS

HTTP (o nosso banco_web hoje)

O Wireshark mostra o **texto puro**: usuário, senha, tudo visível no pacote.

```
{"usuario": "ana", "senha": "123456"}
```

HTTPS (a maioria dos sites reais)

O conteúdo vai **criptografado**. O Wireshark só enxerga IP, hora, tamanho e o nome do site (SNI) — nunca a senha.

```
[dados criptografados]
```

É por isso que o banco_web da aula usa HTTP de propósito: para que a captura seja didática. Um site real, em produção, sempre deveria usar HTTPS.

EXERCÍCIO 4 20 minutos

Capture o próprio ataque

1. Abram 01_wireshark/01_GUIA_PRATICO_WIRESHARK_BRUTEFORCE.md no material_kali.
2. No Wireshark, comecem a capturar na interface de rede correta e apliquem o filtro http.
3. Rodem de novo o bruteforce_simples.py do Bloco 2.
4. No Wireshark, cliquem num pacote **POST** e abram "Follow → HTTP Stream" para ver a senha em texto puro.

Filtro de captura

```
http.request.method == "POST"
```

Isso mostra só as requisições de login, escondendo o resto do ruído da rede.

Filtros úteis para guardar

Filtro	O que mostra
http	Todo o tráfego HTTP (sem HTTPS)
ip.addr == 10.0.0.5	Só pacotes indo ou vindo desse IP
tcp.port == 8002	Só tráfego na porta da API do banco_web
http.request.method == "POST"	Só requisições de envio (login, formulários)

Todos esses filtros — e mais — estão em 01_wireshark/filtros.txt no material_kali.

BLOCO 04

Fechando com Python

O panorama completo: onde o Python aparece em cada frente da cibersegurança

Ofensivo

Python no pentest e no red team

O brute force de hoje é só a ponta do iceberg. Times de pentest usam bibliotecas Python prontas para tarefas parecidas, em escala maior.

Ferramenta	Para que serve
Scapy	Cria e manipula pacotes de rede byte a byte — a base de muitos ataques e testes
python-nmap	Controla o Nmap a partir de um script, automatizando varreduras de rede
Impacket	Fala os protocolos do Windows (SMB, RDP) — muito usada em movimentação lateral
sqlmap	Automatiza a descoberta e exploração de SQL Injection em formulários web
Pwntools	Framework para exploração de vulnerabilidades, muito usado em CTFs

`Defensivo`

Python no blue team e na perícia forense

Do outro lado do muro, Python lê logs, memória e discos para reconstruir o que um atacante fez.

Ferramenta	Para que serve
Volatility	Analisa a memória RAM de uma máquina comprometida em busca de processos maliciosos
Plaso / dfVFS	Reconstroem a linha do tempo de eventos num disco durante uma investigação
PyExifTool	Extrai metadados escondidos de arquivos (fotos, PDFs, documentos)
Paramiko	Automatiza conexões SSH — usada para responder a incidentes remotamente em vários servidores de uma vez

É o mesmo tipo de raciocínio do os.walk() que vimos nas aulas de arquivos — só que vasculhando evidências, não atacando.

Detecção & Dados

Análise de malware e caça a padrões em massa

Analisando malware

Yara-Python classifica arquivos suspeitos por padrões conhecidos; Cuckoo Sandbox executa o malware isolado e registra tudo o que ele faz.

Caçando anomalias

Com Pandas e um pouco de Scikit-Learn, dá para vasculhar milhões de linhas de log e encontrar o padrão estranho que um humano jamais acharia rolando a tela.

É a versão gigante do que fizemos analisando acessos.log na aula de continuidade: achar, no meio de muitos dados, o comportamento que não é normal.

O mapa completo, num só slide

Área	Ferramentas em Python	O que resolve
Ofensivo	Scapy, Impacket, python-nmap, sqlmap	Testar e explorar falhas de rede e aplicação
Forense / DFIR	Volatility, Plaso, PyExifTool	Reconstruir o que aconteceu após um incidente
Detecção / dados	Yara-Python, Cuckoo, Pandas	Classificar malware e achar padrões em massa de dados
Automação geral	requests, Paramiko, os	Conectar ferramentas, arquivos e servidores entre si
Criptografia	cryptography, hashlib, PyJWT	Proteger dados e autenticar de forma segura

Repare: requests e os — os dois que vocês usaram hoje — aparecem escondidos por trás de quase tudo nessa tabela.

O que eu quero que vocês levem desta aula

Vocês não precisam sair daqui sabendo programar. Precisam sair sabendo que **rede + ferramenta + um pouco de script** resolve problemas reais de segurança.

Os três materiais ficam no GitHub de vocês. Voltem a eles quando quiserem — no seu tempo, sem a pressão de uma aula.

O que fizemos hoje

- ✓ Subimos o banco_web em Docker como alvo de teste
- ✓ Rodamos os 3 guias de brute force, do mais simples ao rockyou.txt
- ✓ Capturamos o próprio ataque no Wireshark e vimos a senha em texto puro
- ✓ Vimos até onde o Python vai em cyber — de scripts simples a C2 e malware

Obrigado pela aula. Sucesso no restante do curso!

APÊNDICE · OPCIONAL

Se sobrar tempo: um gostinho do Black Hat Python

Material de consulta — não faz parte do roteiro principal da aula

O material_blackhat: um andar acima

Fica disponível no GitHub de vocês para quem quiser estudar por conta própria — não é conteúdo cobrado nem obrigatório desta aula.

C2 via GitHub

Um programa malicioso que busca comandos escondidos num repositório do GitHub, em vez de falar direto com o invasor.

Keylogger

Um script que registra tudo que é digitado no teclado de uma máquina comprometida.

Detecção de sandbox

Técnicas para o malware perceber se está rodando dentro de um ambiente de análise, e se comportar de forma diferente.

Esse material é baseado no livro *Black Hat Python* e existe para leitura e estudo em ambiente controlado — não para uso fora do laboratório da escola.